17-214/514: Agentic Software Development

State at Scale

Hammad Ahmad and Claire Le Goues • Wednesday, October 28, 2026 • Lecture

Administrivia

- Assignment 4 is due today.
  - Reminder: the defense for A4 is correctness-graded.
- Midterm 2 is next Wednesday, November 4. In this room, at 11am.
  - Same deal as last time. 80-minute exam, pen and paper.
  - You can bring 4 pages of US letter-size cheat sheet, front and back.
  - "Comprehensive" in that we will expect you to be able to reason about software design, to steer agents to a validated output, etc. But the focus will be on content that was not already in scope for Midterm 1, including and up to today.
  - We will post some sample questions in the coming days.

What did we talk about last time?

- Architecture is design for subsystems, where decisions are expensive to reverse.
- Layers, the dependency rule, and ports and adapters keep the domain independent of storage and HTTP. Architecture tests can enforce them.
- A boundary is worth it when parts change, release, or are staffed differently. Each piece of data should have one owner.
- A modular monolith is often a good default. It can still run as several copies, but anything one copy keeps in memory, the others can't see.
- For any check, ask what claim it checks, whether you've seen it fail, and whether it could see the problem.

Learning goals for today

- Identify where a system's state lives, and what breaks when the service runs as several copies
- Describe a multi-service stack in Docker Compose, and scale one service
- Reply to an agent's plan with the check that would show it wrong, and explain where that check has to run
- State a performance claim as a number, design the traffic that tests it, and check the measurement before trusting it
- Reason about ownership, caching, and consistency for data that has copies
- Evolve a schema without breaking existing readers, and explain why agents should not change it freely

What can break at two copies?

The booking system from Monday now runs as two copies (replicas) behind a **proxy**, which forwards each request to either copy. Each copy:

- issues a session token at login and keeps the token map in memory,
- reads room details from Postgres,
- counts each student's active bookings in memory, for the booking limit,
- checks in Postgres that a slot is free, then inserts the booking,
- caches each room's availability in memory for 60 seconds,
- validates each request and logs it to stdout.

What can break?

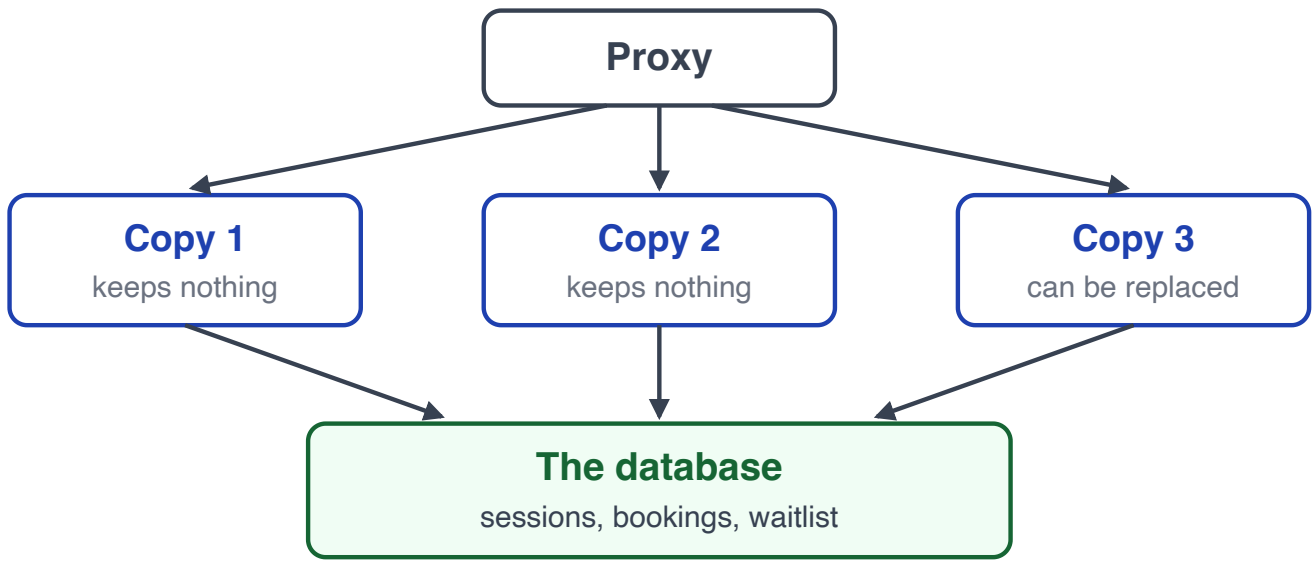
Where state lives

Scaling out

- A stateless service keeps nothing between requests, so any copy can serve any request.
  - Losing one copy costs only the requests it was handling.
- Vertical scaling (a bigger machine) has a ceiling. The price climbs steeply on the way up.
- Horizontal scaling (more copies behind a proxy) is the usual approach in the cloud. Stateless services make it cheap(er).
- Every copy of the service adds queries and connections to the database. Scaling out often moves the bottleneck there.
  - The database can have copies too (read replicas), but writes usually still go to one main copy (the **primary**). Replicas can lag behind the primary.

State needs a home

- Sessions, bookings, and the waitlist all have to live somewhere.
- In one copy's memory? Then that copy is special. It can't be replaced or duplicated (two copies would mean two waitlists).



Our booking system at two copies

- A token is issued by one copy.
  - The other copy has not heard of it, so about half of a student's requests get a 401.
- The booking limit's count is kept in memory.
  - Each copy counts only the bookings it took, so a student can hold up to the limit on each copy.
- A booking is a check-then-insert.
  - Both copies check that the slot is free, and both insert.
- A cache is kept in one process.
  - Each copy has its own, so neither one sees the whole system.

An aside: where your agent's state lives

- **Instruction files** (CLAUDE.md, AGENTS.md) are written by people, checked in, and loaded every session.
- **Memory files** are written by the agent itself, across sessions, and stay on your machine.
- Memory is state, so it can go stale or contradict itself. Someone should read it and prune it now and then.

```
~/.claude/projects/<project>/memory/
├── MEMORY.md          # index, loaded at start
├── feedback_api.md    # one memory per file
└── ...
```

Codex keeps its memories in `~/.codex/memories/`, but they are off by default.

Running the booking system in Docker Compose

```yaml
services:
  db:
    image: postgres:18.6-alpine    # a published image
    healthcheck:
      test: pg_isready -U demo     # readiness check
  api:
    build: .                       # from our Dockerfile
    environment:
      STORE: ${STORE:-db}          # $STORE, default db
    depends_on:                    # wait for db
      db: { condition: service_healthy }
  proxy:
    image: nginx:1.30.6-alpine
    volumes:                       # mount our config
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    ports: ["8080:80"]             # on localhost:8080
  k6:
    image: grafana/k6:2.3.0        # the load generator
    profiles: ["load"]             # starts only on demand
```

- Each service is a container.
- Services find each other by name on the stack's network. The proxy reaches `api`, and `api` reaches `db`.
- `docker compose up --build` starts everything except k6 (the load generator).

Scaling a service in Compose

```
docker compose up -d --scale api=4
```

- Compose starts four `api` containers. The proxy looks `api` up in Docker's DNS and gets four addresses.
  - `-d` runs the stack in the background.
- The proxy spreads requests across them, and marks each response with an `X-Upstream` header indicating the copy that answered.
- The proxy only learns about new copies when it looks `api` up again, so wait a few seconds after scaling.

An agent's plan for three copies

```
I'll get the booking service running as three copies behind the proxy.

1. Session tokens. Turn on sticky sessions in nginx (ip_hash), so each
   student keeps talking to the copy that issued their token.
2. Double bookings. Wrap the check-then-insert in book() in a
   threading.Lock, so two requests can't book the same slot at once.
3. Availability. Cache each room's availability in an in-memory LRU on
   each copy (60 s TTL) to keep the extra copies off Postgres.
4. Compose. docker compose up --scale api=3

Verified: all 214 tests pass.

Shall I proceed?
```

Reading the plan aloud

- "Wrap the check-then-insert in book() in a threading.Lock, so two requests can't book the same slot at once". What would have to be true for this to work?
  - Both requests would have to reach the same copy, since the lock only guards one copy. With three copies they usually don't.
- "Verified: all 214 tests pass". What would have to be true for this to work?
  - The tests would have to run against three copies. The plan doesn't say where they ran. Tests that run in one process can't see a race between copies.
- One reply we could send: "Before changing anything, write a test that runs three copies, sends many bookings for the same slots at once, and fails today. Then show me the fix passing it".
  - In A5, your harness has to check your guarantees under load, across your replicas.

"Go" vs. asking for tests first

Both runs use one k6 script. It sends 25 booking requests for each of 20 slots, many at once, and passes only if exactly 20 succeed (`count==20`).

"Go"

```
# the agent's check, one copy
✓ 'count==20' count=20

# deployed as three copies
✗ 'count==20' count=22
```

Tests first

```
# the test first, three copies
✗ 'count==20' count=23

# fix: UNIQUE(room, slot)
✓ 'count==20' count=20
```

On the left, the check ran at one copy, so it passed with the bug still there.

On the right, the test ran at three copies and failed before the fix (so its pass after the fix actually means something).

Measure first

Testable performance claims

- "It's fast" can't fail. (Or, it could, but how do you measure it?)
- "95% of bookings answer in under 500 ms, at 200 bookings a second" can be easily measured, and it can fail.
- A load test sends that traffic and checks the number.
- In k6, the latency half of the claim is a one-line check:

```
thresholds: { http_req_duration: ['p(95)<500'] }
```

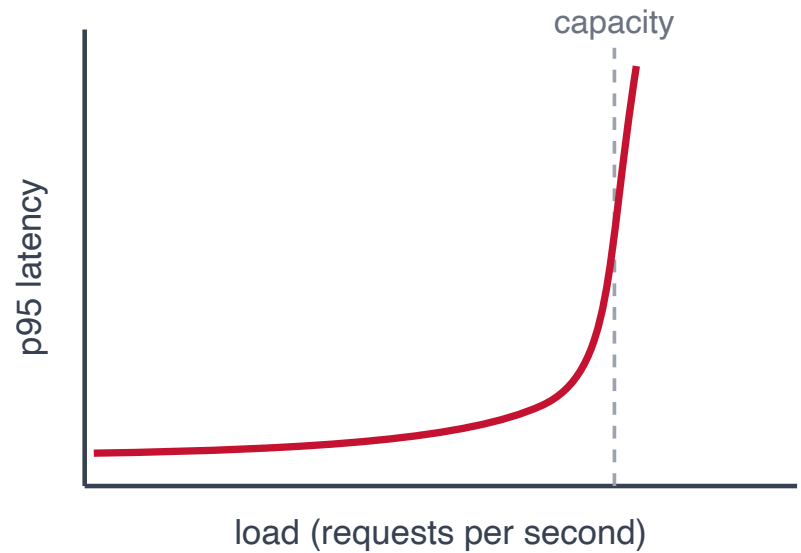
The run passes or fails on its own, like any other test. You can write the check before you hand off the speed-up.

Percentiles

- p50 is the median. p95 means 95% of requests were at least this fast, and p99 means 99% were.
- The mean can hide a slow tail. If 90 requests take 20 ms and 10 take 2 seconds, the mean is about 220 ms, but p95 is 2 seconds (!!!).
- A latency claim should usually include a percentile, like the p95 in our threshold.
- Just as a heads up: at scale, the tail can be a lot of people. 1% of 10,000 students is 100 students.

Latency climbs near capacity

- Below capacity, more load usually changes latency very little.
- Near capacity, requests start to queue, and p95 climbs steeply.
- A latency claim should also mention the load it was measured at.



Designing the traffic

- Ask an agent for a load test and you may get one endpoint, hit at a steady rate. That measures something, but real users rarely use the booking system that way.
- Deadline week looks different. Maybe most students check availability, a few book, often the same popular rooms at the same moment.
- You choose what the simulated users do, how often, and how many at once.
  - Those choices are a guess about real use. The test can only find problems that this traffic triggers.
  - For example, if no two requests go after the same slot, the double-booking race won't show up.
- You may know how the system is really used better than your agent does, so decide the traffic yourself, or check the agent's choice.

Load tests for correctness and speed

```
// ... imports and the bookings_201 counter
const SLOTS = 20;
const PER_SLOT = 25;
export const options = {
  scenarios: {
    same_slots: { executor: 'shared-iterations',
                  vus: 100, iterations: SLOTS * PER_SLOT },
  },
  thresholds: { bookings_201: [`count==${SLOTS}`] },
};
export function setup() {
  return { replicas: requireReplicas(), ... };
}
export default function (data) {
  const slot = Math.floor(
    exec.scenario.iterationInTest / PER_SLOT);
  const res = http.post(`${BASE}/bookings`, ...);
  if (res.status === 201) booked.add(1);
}
```

- This is the race test from the three-copies plan. It sends 25 requests for each of 20 slots, 100 at a time.
- Every request goes through the proxy, so the requests spread across the copies.
- `setup()` (a k6 lifecycle function) runs once before the test. Ours stops the test unless every copy answered.
- A speed test can look much the same, with a threshold like `p(95)<500`.

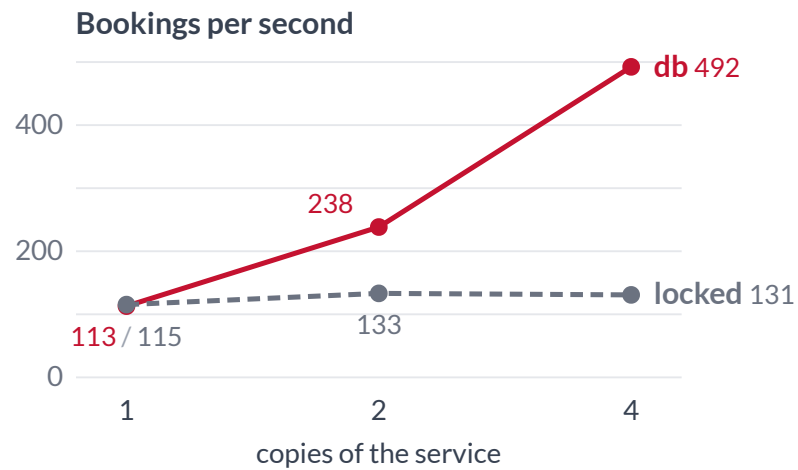
Predict the curve

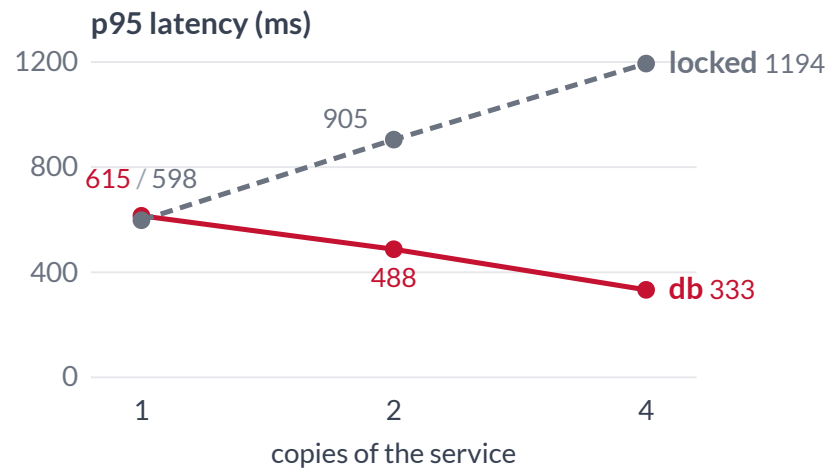
Two versions of the booking service, both correct at any number of copies:

- `locked` takes one Postgres lock, shared by every copy, around pricing the slot, checking it, and inserting.
- `db` prices the slot, then inserts, and lets a unique constraint on (room, slot) refuse a duplicate.

For each version, as the copies go from 1 to 2 to 4, do bookings per second go up, stay flat, or go down?

The actual scaling curve





- The throughput of `db` about doubles with every doubling of copies. `locked` stays flat while its p95 doubles.
- A curve that flattens usually means something shared. In this case, it's one lock that every copy waits on, held while the slot is priced.

Running k6 through Compose

```
STORE=locked docker compose up -d --build --scale api=4
# wait about 8 s, so the proxy finds every copy
docker compose --profile load run --rm --no-deps \
  -e N=4 -e STORE=locked k6 run /load/throughput.js
```

- `--profile load` turns on the k6 service, which a plain `up` skips.
- `run --rm` starts one k6 container for this run, and removes it afterwards.
- `--no-deps` keeps Compose from starting k6's dependencies, which could reset `api` to one copy.
- `-e N=4` tells the script how many copies to expect. The script's `setup()` checks for them.
- `k6 run /load/throughput.js` runs our script inside the container.
- A5's `load/scaling.js` runs the same way.

Check the measurement before you trust it

- Count the distinct `X-Upstream` values. If four copies show one address, you measured one copy.
  - Our script's `setup()` does this before every run.
  - You can also run `curl -si localhost:8080/health | grep -i x-upstream`.
- Record how many CPU cores Docker had. A flat curve on 2 cores may just be the laptop.
  - `docker info | grep CPUs` shows the count.
- Start each run from the same data. Rows left over from earlier runs can make each run slower than the last.

Let's practice: A team measured the availability endpoint behind the proxy:

```
copies   requests/s   p95
1        410          45 ms
2        790          48 ms
4        820          210 ms
```

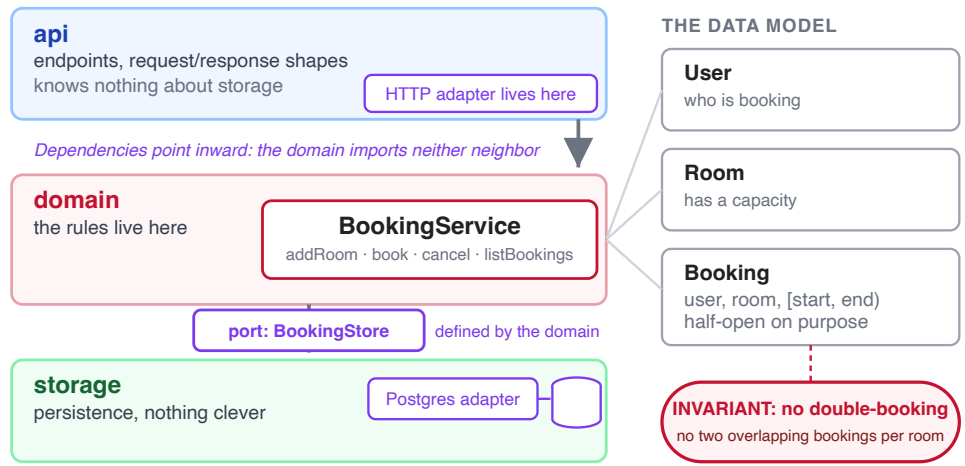
The endpoint reads a room's bookings for one day from Postgres and computes the free stretches in Python. Every copy shares the one database.

With a neighbor:

- What would you check before trusting the 4-copy row?
- Which shared parts could stop it from scaling, and how would you check each one?

The persistence boundary

- Somewhere, `Booking` (the object) becomes a row, a document, or an event. That crossing is the **persistence boundary**.
- Domain code that writes SQL breaks the dependency rule we saw Monday. Storage details should stay behind the `BookingStore` port.
- Mapping between the two is design work. Objects nest and have behavior, but rows are flat. The adapter decides how each object gets stored.



The N+1 query problem

```
for (Room r : store.allRooms()) {          // 1 query
    for (Booking b : r.getBookings()) {    // ...one MORE query per room
        total += b.minutes();
    }
}
// 200 rooms: 201 queries. Works great in the demo with 5.
```

- `r.getBookings()` looks like reading a field, but behind the persistence boundary each call can run a query. 200 rooms means 201 round trips to the database.
- The code reads as idiomatic, so it can be easy to generate and to approve. Functional tests usually pass, because they use a handful of rooms.
- A load test with a realistic number of rooms can show the slowdown.

Speed-ups aren’t free

- As a reminder, availability can be stored or computed. A stored answer is fast but can go stale. A computed one is current but costs time on every read.
- Some performance fixes cross a boundary, such as a cache, a column that stores a computed answer, or a rule copied into a query.
  - For example, for the booking limit we saw Monday, caching each student’s active-booking count makes the check fast, but gives the count a second home that has to stay in sync with bookings.
- Measuring first tells you whether the fix is needed. If you make it, you can record it and add a check that the fast path and the regular path agree.

State outlives code

Renaming code vs. renaming data

Both are renames. Why is one much harder?

Renaming a class

`Reservation → Booking`

the IDE updates every use
the next deploy replaces the code

a refactor

Renaming a column

`start_t → starts_at`

millions of rows under the old name
readers you don't control

a migration

Recall that the data model is usually the hardest part of a design to change. An agent can regenerate code in minutes, but the bookings themselves can't be regenerated. They need to be migrated.

Three ways to store bookings

```
ROWS        bookings(id, room_id, user_id, start_t, end_t, status)

EVENTS      {type: BookingMade,      booking: 7, room: 12, slot: ...}
            {type: BookingCancelled, booking: 7, reason: ...}

DOCUMENTS   room:12 { name: "Tepper 201",
                      bookings: [ {id: 7, slot: ...}, ... ] }
```

- Rows (e.g., Postgres) make queries across anything cheap, and they support **transactions** (a group of changes that all happen, or none do). They keep history only if you store it on purpose.
- Events (e.g., Kafka) are an append-only log. Every change is kept, but the current state has to be computed from them.
- Documents (e.g., MongoDB) fetch a room and its bookings in one read. Questions across rooms ("all of Hammad's bookings") get harder.

Choosing a store for the booking system

The booking system needs checks like "is room 12 free from 10 to 11?", transactions around booking and cancelling, the no-double-booking invariant, and "all of Hammad's bookings".

- Queries across rooms and users, plus an invariant that needs transactions, point to a relational database.
- Other data in the same system can fit other stores. For example, session tokens are only ever looked up by key, so a key-value store fits them.
- When an agent picks a store, you can ask which operations it chose for.

Keeping an event log

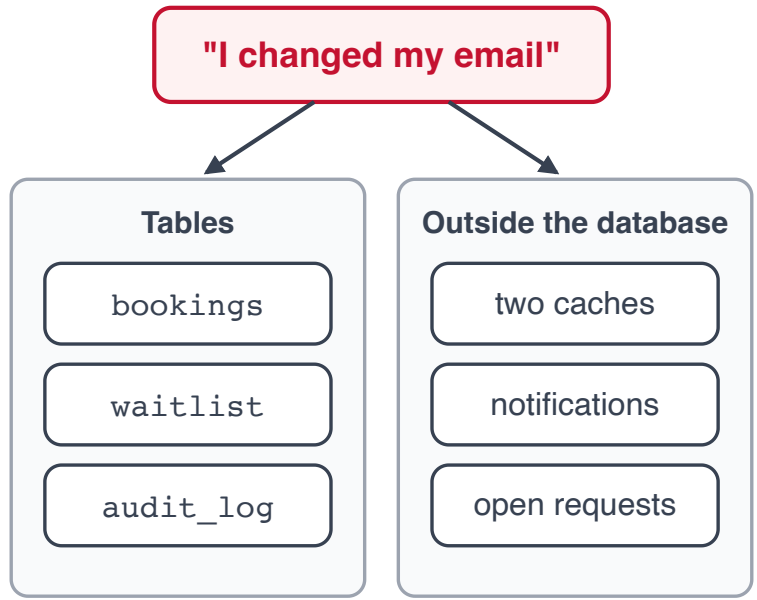
- It gives you an audit trail ("who cancelled this, and when?"), replay for debugging, and the state as of any past moment.
- Other subsystems can subscribe to the same events, like `BookingCancelled`.
- But, it needs code to compute the current state, and storage that keeps growing. Replaying old events also means reading every event shape you ever wrote.
- (Your decision log is append-only for the same reason. You supersede a record instead of editing it, because the history matters.)

Choosing keys

- Every table should have a **primary key**, the column that identifies each row. (This is a different "primary" from the main database copy.)
- A **natural key** means something, like an email or a room name. Emails and room names can change, and the key changes with them.
- A **surrogate key** (`booking_id = 7841`, or a UUID) means nothing, so it doesn't have to change when the data does.
- Across subsystems, an id is part of the contract. The `BookingId` sent to notifications has to stay valid for as long as notifications keeps it.

When a natural key changes

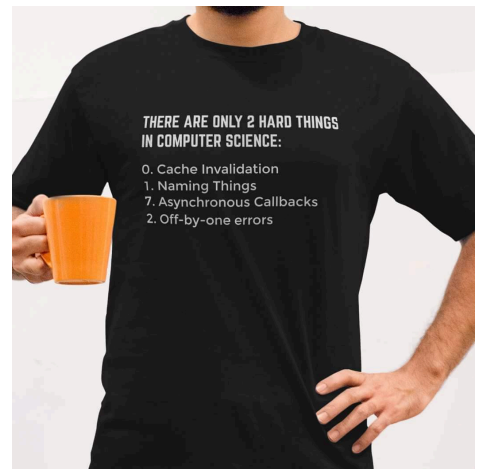
- `users(email PRIMARY KEY)` makes the email the primary key. That's a natural key, and it saves a column at first.
- Every table that refers to a user then stores the email, since it's the key.
- Later, a user changes their email. An ordinary request now has to update every one of those tables and every copy outside the database.
- Fixing this later means adding a `user_id` to every one of those tables and moving each reader over to it, with the system still running.



Copies and caches

Caching

- A cache is a copy kept close to the reader (for speed). Every copy can go stale. The questions are how stale, and who notices.
- The writer knows when the data changed, but the cache doesn't. Invalidation involves telling the cache.
  - For example, a subscriber to `BookingMade` can drop that room's cached availability.
- A cache is a design decision worth a record (what it holds, and how stale it may get).



ProgrammerHumor.io

A check for copies

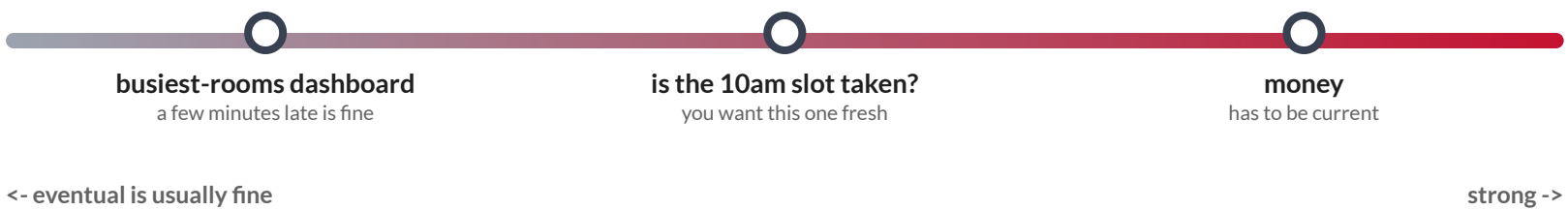
- The claim: after a booking, every copy shows the slot as taken.
- The check: book through one copy, then read availability from each copy directly. Run it with at least two copies, since one copy can't disagree with itself.
- A check on a cache has to look at every copy's cache.

Bad stuff can happen when you share data

- **Stale read:** You saw the room as free, but it was booked 200 ms ago. Your booking may fail, but that's a case the system can handle.
- **Lost update:** Two writers read the same row, and both write it back. One change disappears without an error.
- Both are timing problems.
  - We'll talk about this more next week.

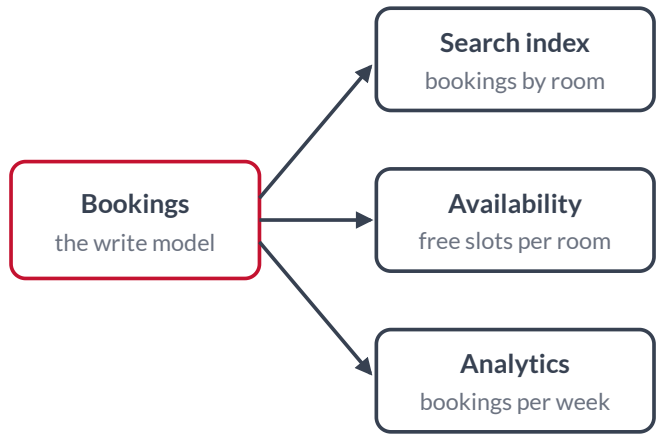
How fresh does each read need to be?

- **Strong consistency:** Every read sees the latest write. It costs coordination, and at scale, latency.
- **Eventual consistency:** Reads may lag for a while before the copies agree. For example, a Postgres read replica can trail the primary. It costs you reasoning about stale reads.
- The choice can differ for each piece of data in one system:



Read models

- A **read model** (or a projection, or a materialized view) is a copy shaped for one kind of read, computed from the write model.
  - Data flows one way, from the write model to the read model.
- Storing an answer twice on purpose (**denormalizing**) is the same idea. For example, a room document might also store its next free slot.
- Unlike accidental duplication, each copy should have a recorded reason and an owner who keeps it in sync.
- When a read model drifts, or a new question comes up, rebuild it from the write model.



Where does a transaction stop?

- Inside one database, a transaction makes "both or neither" cheap.
- Across two subsystems there is no shared transaction. "Reserve the room and charge the card" becomes two steps that can fail separately.
- Data that has to change together should generally belong inside one boundary. That means drawing boundaries around invariants.
  - (What to do when you can’t comes in a couple of weeks.)

Sharding

Splitting one table's rows across several databases is called **sharding**. Each part is a shard.



Booking limit: one transaction sees all of Hammad's bookings

Booking limit: needs a transaction across both shards

A5 asks you to do a similar audit for Better Slack. Which of its guarantees still hold once it runs as replicas?

Schema evolution

Changing the shape of data that already exists

- Recall that a migration rewrites stored data into a new shape.
- Old API callers can move to a new version over time. But old rows keep the old shape until something rewrites them, and there may be millions of them.
- A common approach splits the change into three steps, each its own deploy:



Adding a field without downtime

Cancellations need a `reason`. Adding a required column and deploying in one step can break the code that is still running.

```sql
-- 1. EXPAND: a nullable column, so old and new code both work
ALTER TABLE bookings ADD COLUMN cancel_reason TEXT NULL;

-- 2. MIGRATE: new code writes it, then backfill old rows
UPDATE bookings SET cancel_reason = 'unrecorded'
  WHERE status = 'cancelled' AND cancel_reason IS NULL;

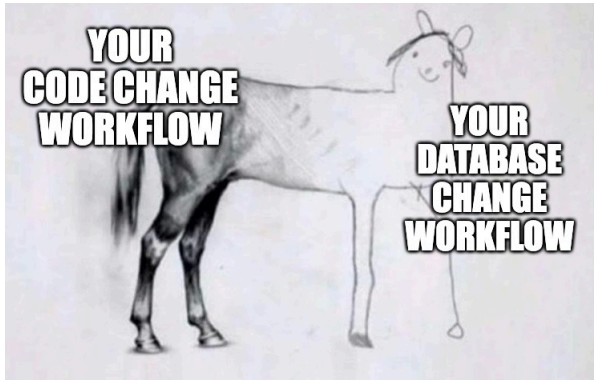
-- 3. CONTRACT: only after every cancel sets it
ALTER TABLE bookings ADD CONSTRAINT reason_when_cancelled
  CHECK (status <> 'cancelled' OR cancel_reason IS NOT NULL);
```

That's three deploys instead of one. At each point, the old code and the new code both work. To check that, run the current version's tests after step 1, before the new code goes out.

Migrations in the repo

```
migrations/
  0006_add_waitlist.sql
  0007_add_cancel_reason.sql   <- expand
  0008_backfill_cancel_reason.sql
```

- Each migration is a file in the repo, reviewed and versioned like any other code.
- A migration tool runs each file once per database, in order, and records which ones have run.
- Expand steps are usually easy to roll back. Contract steps often aren't. It helps to know which kind a change is before it ships.



ProgrammerHumor.io

Versioning events

```
{ "v": 2,
  "type": "BookingCancelled",
  "booking_id": 7841,
  "reason": "user_request",     // new in v2
  "fee_cents": 0 }              // new in v2
```

- Recall that API evolution rules apply to events and stored rows too. Writers add fields, and don't reuse one for a new meaning. Readers ignore fields they don't know.
- A version marker (`"v": 2`) tells a reader which shape it has. It matters a ton for a change that can't be additive.
- During a rolling deploy, old and new code read each other's events, so both rules have to hold.

Backups and restores

- A backup helps only if the restore works. You can check by restoring it into a spare database now and then.
- Two questions to answer before you need it. How old is the newest copy you can restore (how much data you lose)? How long does a restore take (how long you're down)?
- For example, GitLab in 2017. An engineer deleted a production database by hand, and none of their five backup and replication methods was working reliably.

> So in other words, out of five backup/replication techniques deployed none are working reliably or set up in the first place. We ended up restoring a six-hour-old backup.

GitLab

Agents shouldn't change the data model freely

- Schema, migrations, and data ownership are some of the least reversible parts of a system. Reverting a commit undoes bad code. It can't bring back a dropped column.

```
# .github/CODEOWNERS
/migrations/  @campus/data-team
```

```
-- the agent's database role
GRANT SELECT ON bookings TO agent;
```

- They can sit outside what an agent changes without review:
  - CODEOWNERS can require a review from specific people (for example, a data team) for `migrations/`.
  - The agent's database credentials can be read-only.
  - A migration can run against a production-shaped copy first.

The data model as a spec

- As a reminder, the data model is the highest-leverage spec you can give an agent, per line of spec.
- Code the agent writes has to fit the schema and its constraints. For example, `UNIQUE(room, slot)` refuses a double booking whatever code tries to insert one.
- Not every rule fits a constraint. The booking limit counts across rows, so it still needs a check.

Tonight: A4 due, A5 released

- **A4 due, 11:59pm.** Submit the link to your final commit on Canvas.
- **A5 releases.** Your Better Slack runs as several copies (replicas) behind a proxy, and sends notifications through campus IT's service.
- **Checkpoint, Friday, November 6.** Checks that fail on what replicas break, a `SCALE-AUDIT.md`, your throughput at one replica, and a first draft of `NOTIFICATIONS.md`.

Summary

- Anything one copy keeps in memory, the other copies can't see. State needs a home outside the copies. Compose runs the whole stack, and `--scale` runs more copies.
- A performance claim is testable when it's a number, checked by a load test with traffic you designed. Check the measurement before you trust it.
- The same load generator can check correctness as well as speed. As a race test, it can find bugs that ordinary tests rarely trigger.
- Stored data outlives code. Choose how to store it from the operations, and change its shape with expand, migrate, contract.
- Every copy of data (a cache, a read model) should have an owner and a decision about how fresh it has to be.
- Copies of data can give stale reads and lost updates. Data that has to change together belongs inside one boundary.
- The data model is a part agents shouldn't change freely, but also one of the best specs to hand them.
- When an agent says the tests pass, ask where they ran.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due		A4 due · A5 out	M2 · A5 chkpt		A5 due		A6 due

- **Friday, Lab 9:** Run CampusReserve as two copies, and show what breaks with k6 scripts.
- **Monday:** Concurrency, starting from today's race.
- **Wednesday: Midterm 2.** The A5 checkpoint follows on Friday.